Contents

1 CabiNET

Welcome to the CabiNET monorepo! All project code lives in here, broken into multiple subdirectories:

- **/app**: mobile app
- **/backend**: backend web/api server
- **/firmware**: firmware that runs on the actual pill organizer (temporarily moved out of monorepo)

See architecture for a general overview of how the project works.

1.1 Subcomponent Overview

Please view the appropriate pages for in-depth documentation on each component.

1.1.1 App

The mobile app is written in Dart using the Flutter framework. Primary rationale for picking this stack was for ease of cross-platform development, relative maturity of the platform, and quality community extensions.

The app is generally "dumb". It is a window and a conduit to the backend. All information shown in the app comes directly from the backend server, even if pill organizers are connected

to Bluetooh. Information is pulled from the server via HTTP, and "live" data is achieved via polling.

See app documentation.

1.1.2 Backend

The backend server hosts the API, which physical pill organizers and the mobile app interact with. It serves as the "source of truth" for all devices, and records all events, manages scheduling, and more. It is written in Java using the Micronaut framework. Micronaut was picked because it is lightweight, very fast, and enables rapid development of APIs. Persistence is provided by `micronaut-hibernate-jpa`. The database is PostgreSQL.

The backend is currently hosted on Heroku. As of writing, effort is being made to move to fly.io for production, and use Heroku for staging/testing.

See backend documentation.

1.1.3 Firmware

The firmware is the software that runs on the pill organizer devices. It handles all of the business logic that runs on-device, such as lighting LEDs and tracking bins.

See firmware documentation.

1.2 Project Status

The entire CabiNET codebase is currently transitioning from rapid prototyping to production. Many components were written as prototypes while requirements were still changing and the vision for the product was in flux. There is a general effort project-wide to bring components into production-ready status through refactoring, but at this time quality may be mixed.

2 Architecture Overview

The backend acts as a data broker between pill organizers and the app. It is responsible for maintaining all data, and ensuring the system is safe and consistent.

2.1 How state is managed

The state of a pill organizer is distributed between the firmware and the backend. The backend maintains the authoritative state - generally it can override whatever the firmware thinks the state is in. However, a key requirement of this project is that the pill organizer continues working while completely disconnected from WiFi and Bluetooth. Thus the firmware maintains its own state as well, and a sync procedure keeps the two consistent.

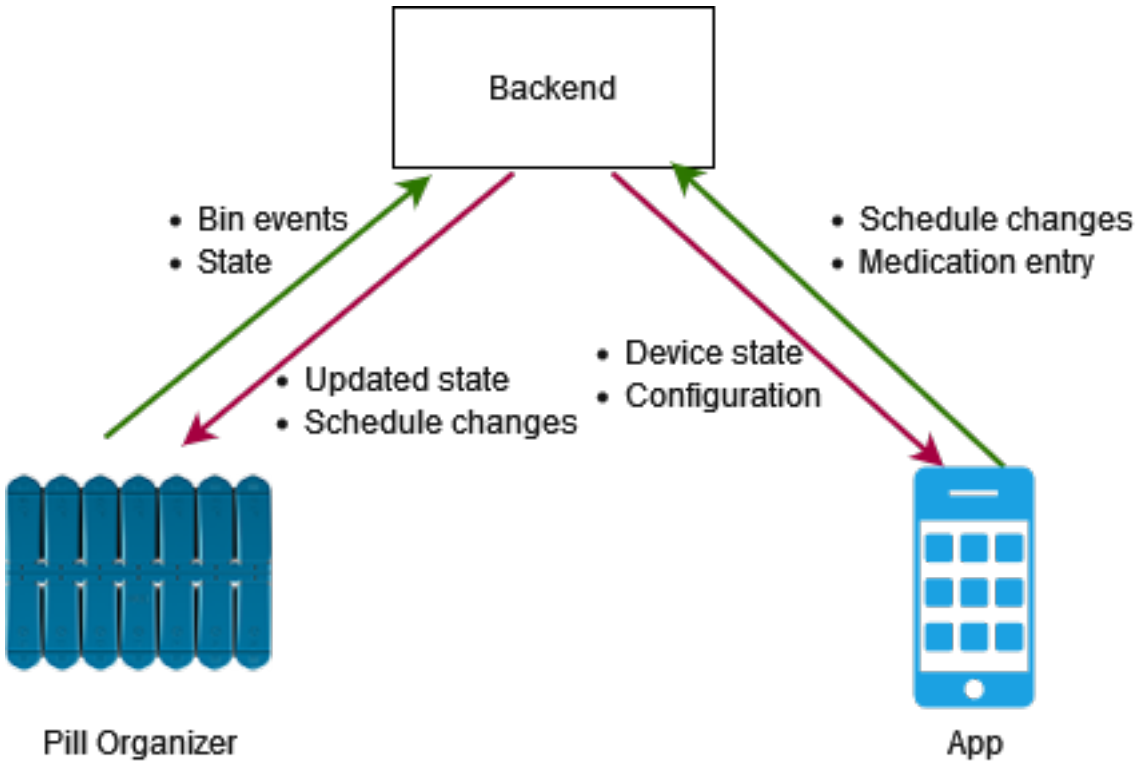

Figure 1: architecture diagram

2.1.1 What is state?

We consider the state of a pill organizer to be the operational data of the device. State determines which LED should be lit on a particular bin (red/green/flashing etc.) and when a transition should occur. State transitions are also used to dispatch notifications to users that they need to take their medication.

The device state is divided into 14 "**bin states**", referring to each physical bin on a pill organizer. Since each bin operates more or less independently, the entire device state is simply the collection of these 14 bin states. In other words, the device state is an array of 14 bin states, ordered by bin ID.

We can define a bin state as a pseudocode structure:

```
struct bin_state {
    timestamp   scheduled_time;
    bin_status  status;
}

enum bin_status {
  DISABLED = 0;     // This bin is not scheduled
  TAKEN = 1;        // The medication in this bin has been taken already
  MISSED = 2;       // The medication in this bin was not taken as scheduled
  PENDING = 3;      // The medication in this bin should be taken at some time in the fu
```

```
  TAKE_NOW = 4;     // The medication in this bin should be taken now
}
```

`scheduled_time` is the time when medication in this bin should be taken and is a Unix timestamp (seconds). `status` indicates the operating status of the bin and generally reflects what LED is shown on device.

2.1.1.1 In the code As of writing, state structures are defined in **three** places in the project:

- **firmware/proto/pill.proto**: Protocol buffer definition for wire serialization.
- **firmware/include/pill_types.h**: C structure for use within the firmware.
- **backend/src/main/java/jct/pillorganizer/model/device/DeviceState.java**: Java model for persistence.

There is effort to move these definitions to a single place for easier maintenance.

2.1.2 State hash

Because state is distributed between the firmware and backend, a way of quickly checking if two device states are the same is needed.

Instead of manually comparing every field of the state, a CRC32 (little-endian) hash is computed over the fields of the state. If the hashes match, it is safe to assume that the two states are the same.

The hash is computed using the following algorithm. Order the bin states by bin ID. For each bin state, digest the status as an 8 bit integer and digest the timestamp as a 64 bit integer. The hash at the end is the state hash.

2.1.2.1 In the code

- firmware/src/pill_state.c (`state_build_sync_request`)
- backend/src/main/java/jct/pillorganizer/device/DeviceStateWrapper.java (`calculateStateHash`)

2.2 How syncing works

A sync uploads data from the device to the backend and is *always* device-initiated. The device initiates a sync at a certain frequency (currently roughly 20 second intervals) or after an event is triggered. A sync is two-way. First, the device uploads its state to the server, and the server replies with its state. The device will always override its own state with the server, as the server is authoritative.

A sync is an HTTP PUT request to `/api/v1_2/device/sync` (direct device WiFi sync) or `/api/v1/device/{id}/sync` (app-proxied Bluetooth sync). The request content is the `SyncRequest` protobuf structure and the response content is the `SyncResponse` protobuf structure.

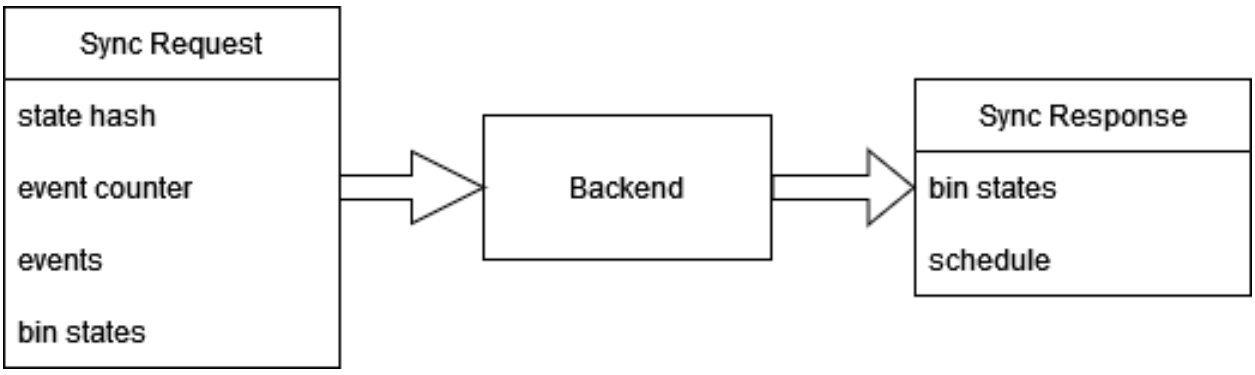

Figure 2: two-way sync

The state hash allows the server to short-circuit the sync process if the states are already the same. The event counter is intended to be a counter of the total number of events triggered on the device in its lifetime, however as of writing this behavior may be non-functional. The events field contains all events recorded on device *since the last sync*. The bin states field contains the device state.

The sync algorithm is as follows:

1. Process all events in the request (see below).
2. Update event counter and state hash in the backend's Device record.
3. If the backend is 28 events behind (i.e., over 28 events have occurred since last sync), accept the device's state as truth, and update the backend state to match the device's.
4. Respond with the backend's state and the device's schedule.

See backend/src/main/java/jct/pillorganizer/device/DeviceStateWrapper.java (`sync`).

2.3 How an event is recorded

Events are things that happen on a device that we care about. We consider two events: a bin has opened, and a bin hasn't been opened when it is scheduled. There used to be a third event (bin closed) however this is deprecated (though there might be references to it around the codebase). Note that we use "bin closed" as our bin open event, since it fires after the user has finished using the bin.

Events are stored and processed on device and are then synced to the backend for persistence. Since the device must function in offline mode, event processing is replicated in **both the backend and the firmware**.

Missed events are handled by simply updating the appropriate bin status to missed. Bin open events are handled with the following process:

- If the bin status is "take now", the bin moves to "taken".
- If the bin status is "pending" or "missed":
  - If the bin was opened between the previous bin and the next bin, the bin moves to taken.

  - If the bin was opened after the previous bin but there is no next bin, the bin moves to taken.
  - If the bin was opened before the next bin but there is no previous bin, the bin moves to taken.
  - In all other cases, the event is recorded but the bin maintains its state.
- All other statuses, the event is ignored.

All events are persisted into the database.

2.4 How scheduling works

Right now, only a simple schedule can be programmed onto a device. A simple schedule uses the same times every day, one for the morning bin and one for the evening bin. Some groundwork has been laid to support more complicated schedules, but that has not been implemented at this time.

The schedule of a device is stored as an array of 14 schedules, one for each bin. A bin schedule can be defined with the following pseudocode structure:

```
struct bin_schedule {
    uint32      seconds_from_00;
    DayOfWeek  dayOfWeek;
}
```

The day of week indicates which day of the week the bin is scheduled on, whereas the `seconds_from_00` indicates when on that particular day the bin should be opened, in day-epoch format.

This structure exists in the firmware (`bin_schedule_t`), the backend (`DeviceSchedule`) and in the protobufs (`BinSchedule`).

When a user updates their simple schedule, the following things happen:

1. All DeviceSchedule entries for the device are updated with the new times.
2. The state is rebuilt (see below).
3. The next time the device syncs to the backend, the backend replies with the new schedule and state.
4. The device updates its internal bin schedule and rebuilds the state.

2.4.1 Status transitions

Status transitions are when a bin state moves from one status to another due to some time having passed. One moves bins from "pending" to "take now" when the scheduled time of that bin has passed. The other moves "take now" to "missed" if the bin isn't opened within **10** minutes of its scheduled time. We consider the 10-minute miss interval to be the "**missed dose threshold**". Code to handle these transitions are found in both the firmware and backend.

2.4.2 State rebuilds

Since a bin state contains a "scheduled time" field, this field can get out of date when the user changes their schedule. State rebuilding synchronizes the scheduled time fields with the device's schedule.

The rebuild algorithm loops through all bin states that are *not* taken or missed (that is, are disabled, pending, or take now). The bin's scheduled time is updated to the new time. If the new scheduled time is after the missed dose threshold, it is set to pending. Otherwise, it is set to disabled.

This rebuild function exists in both the backend (`DeviceStateWrapper`) and firmware (`state_rebuild_schedule`).

2.4.3 Pill Reloading

Pill reloading is handling when the user fills their pill organizer with pills again at the end of the week. The pill reload procedure is very simple. The device state is reset, so all bins are disabled and their scheduled time cleared. Then the state is reset.

We are left with a "clean slate" and the device is ready to be used for another week.

(this process has a number of limitations and needs to be revisited)

2.5 How provisioning works

We use the Wifi provisioning system built in to the esp-idf, but we use a couple of custom endpoints to transfer our application-specific data. On the app side, we use a custom forked version of flutter_esp_ble_prov with a number of patches.

1. The app scans for BLE devices. For best support between iOS and Android, we look for devices starting with "PROV_" (change this).
2. When a matching device is found, we scan for WiFi networks (handled by idf). Present the user with a list of networks.
3. Call the `serial-no` custom endpoint to get the device's serial number.
4. Start provisioning with the backend, using the serial number to get an OOB key. This is a shared key between the backend and firmware, used for authentication.
5. Call the `provision-key` custom endpoint, writing the OOB key to the device.
6. Call into the idf's provision function, writing the WiFi credentials to the device.
7. When the device connects to WiFi, it makes an HTTP request to the backend indicating that provisioning was successful.
8. The app polls the provision verify API endpoint until success.

NOTE: the firmware uses ESP provisioning in "security 0" mode. We need to use a secure mode and figure out how to distribute POP keys before production.

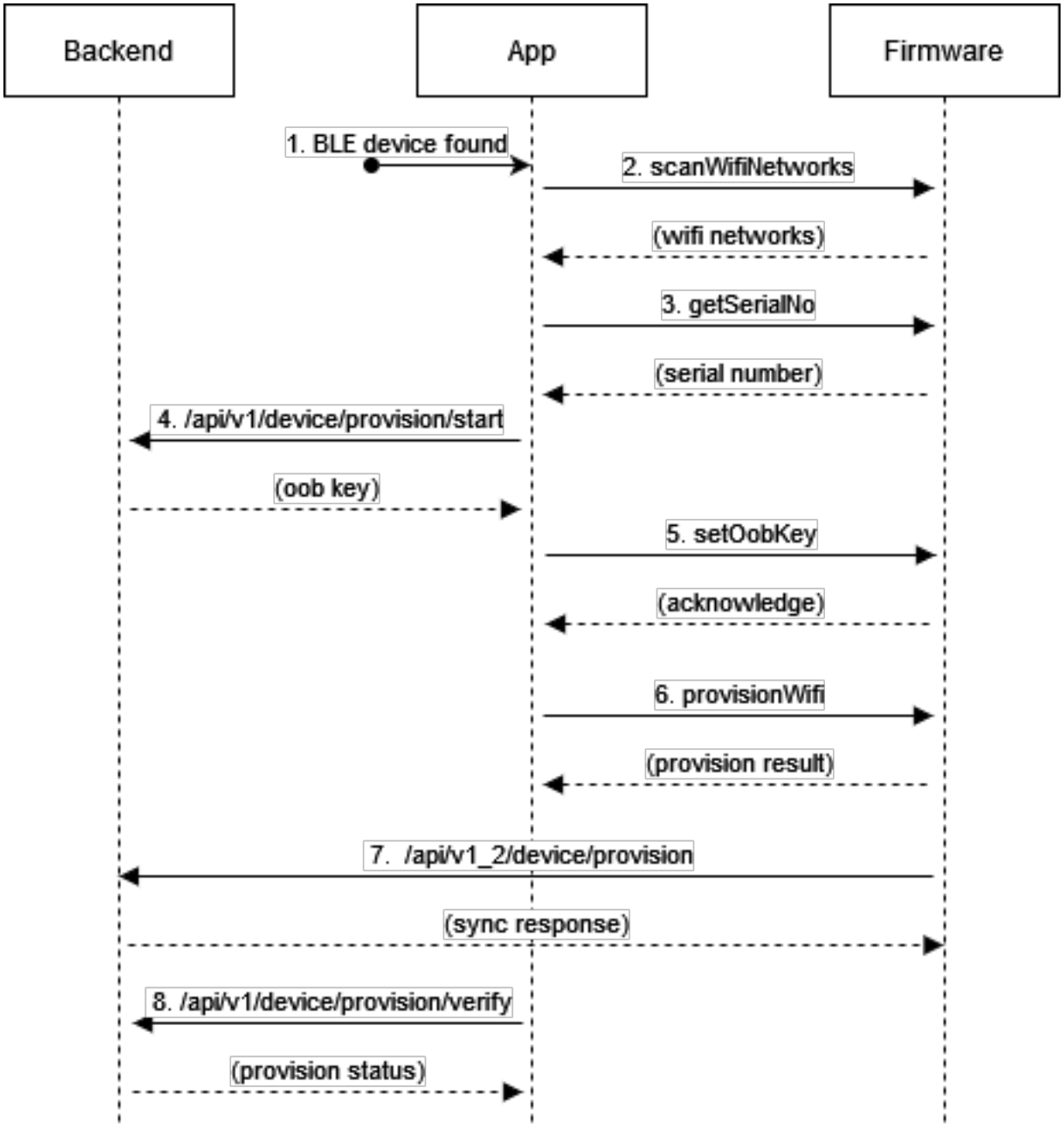

Figure 3: provisioning flow

3 Common Concepts

3.1 Bin IDs

Each bin on a device is assigned a bin ID. The Monday PM bin is considered bin 0, and Sunday AM bin 13.

	MON	TUE	WED	THU	FRI	SAT	SUN
AM	0	2	4	6	8	10	12
PM	1	3	5	7	9	11	13

3.2 Day-Epoch (Seconds from 00)

This project needs to deal with time a lot. Not time as in datetime, but just time... as in 8 o'clock, or 3:23. That is to say, time without regards to the date. Java and Dart have some utility classes for this, but the firmware doesn't. This necessitated a more low-level time format. You may see it referenced as **day-epoch** or **seconds from 00**.

It is simply the number of seconds since midnight.

For example, 2:24:30 AM in day-epoch format is computed like so:

```
   (3600 * 2 )     // convert hours to seconds
 + (60   * 24)     // convert minutes to seconds
 + 30              // remaining seconds
 = 5070
```

Note that timezone is not specified. In fact, timezones wouldn't work with this format, because a time in one zone could be a different day in another. That's something that can't be easily dealt with, so the time is treated as zoneless.

3.3 Serial numbers and device IDs

The **serial number** of a device is derived from the MAC address, which *should* be unique on a per-device basis. We take the MAC address with `esp_efuse_mac_get_default` and pad it with two zero bytes to make a 64-bit integer. The backend stores serial numbers as integers but serializes them as HEX strings. In the protobufs, they're serialized as a 64-bit integer. As of writing, there is some discussion about using something other than MAC address, but nothing has been decided.

Device IDs are sequentially assigned numbers (Postgres sequences) in order of when they are created (provisioned for the first time). In that sense, a device ID is a backend concept. The firmware is unaware of its device ID, it uses its serial number for that purpose.

3.4 Building this documentation

The documentation for this project is sourced from both Markdown files located in `/docs` and automatically (JavaDoc and Swagger). To "compile" the markdown into a PDF, pandoc is used. Run `/docs/build.bat` to compile the markdown files into `docs.pdf`.

To compile the backend documentation (generated from javadoc-style comments), Doxygen is used (notably *not* javadoc, since we want to generate a PDF). We use the LaTeX backend, which outputs a .tex document, which must be compiled into a PDF (see `/docs/latex/buiild.bat`, you need MiKTeX or another Latex distribution).

The API documentation is generated from the backend's OpenAPI annotations. The PDF is generated using RapiPDF. Make sure you've compiled with Maven (needed to generate the swagger files) and run the backend, and visit http://localhost:8080/swagger-ui/. There should be a button in the top left to download a PDF.

4 Backend

The backend API server is written in Java using the Micronaut framework.

It is a web server, but only serves API requests (there are no "views" or templates).

WARNING: the backend has not gone through an exaustive security audit of all endpoints and thus is **not production ready**.

4.1 Database

PostgreSQL is the main database for this project. Effort was made to avoid Postgres-specific features but there's no strong rationale anymore. Generally, constraints are avoided to enable more rapid development.

The backend only interacts with the database through the framework's Hibernate integration. The Hibernate session is never used directly, only through the framework's repository classes.

Migrations are handled by Flyway with the files in `resources/db.migration`. They can create the schema on a clean database.

4.2 AuthN & AuthZ

(see the section below)

Authentication is provided by Micronaut Security. The backend has 3 different authentication providers for each user type.

BCrypt is used for password hashing (see AuthService).

We also use annotation-based ABAC to authorize a specific user to a specific resource. There is currently only one such annotation: `@DeviceABAC`. This annotation ensures that the currently logged-in user has access to a specific device.

4.3 Device State Transitions

(see `DeviceStateJob`).

The backend is responsible for monitoring **all** bin states for every device in the system. To send accurate push notifications and to accurately manage state on the backend, the backend transitions bin state statuses when appropriate.

Every ten seconds, the following is run:

- Bin statuses that are "PENDING" that are due to be taken are moved to "TAKE NOW".
- Bin statuses that are "TAKE NOW" and it's been longer than 10 minutes since scheduled are moved to "MISSED".

Each one of these state transitions fires off a push notification.

Note that because there is no mutual exclusion on the state job, the backend is unsuitable for more than one instance running at any given time. This must be fixed before production.

4.4 Building, Deploying, and CI/CD

The backend uses the Maven build system. The backend can be built (for testing) with `mvn package`.

4.4.1 CI/CD

Github Actions are used for CI/CD, located in the `backend.yml` workflow file. It is configured to automatically build and deploy the backend to production on every push to the Github repository. Each release is tested to make sure it compiles and passes some automated regression testing.

4.4.2 Deploying

Micronaut supports Docker packaging, which is quite convenient. We use the default configuration Micronaut provides for us, so we don't have a Dockerfile. A Docker image is built with `mvn --batch-mode --update-snapshots package -Dpackaging=docker`. We push this Docker image directly to the cloud provider.

4.4.2.1 Environment Variables A number of environment variables are required for operation.

- DB_DATABASE
- DB_HOSTNAME
- DB_PASSWORD
- DB_PORT
- DB_USERNAME
- FIREBASE_CLIENT_EMAIL

- FIREBASE_PRIVATE_KEY
- FIREBASE_PROJECT_ID
- PORT (port to bind HTTP)
- SIGNING_JWK (used to protect JWT)

Secrets are typically stored in the hosting provider's secret store.

4.4.3 Automated Testing

Put simply, the backend is not *unit* tested. There are, however, a battery of automated tests against certain components known to cause breakage and regressions. Maven Surefire kicks off a Spock spec test (currently there's only one: UserOnboardSpec). The tests use an HTTP client and call the HTTP endpoints directly.

Testcontainers are used to automatically spin up a real Postgres database on the testing machine to test against. All of this should happen automatically by Maven.

4.5 Consuming the API

The API has auto-generated OpenAPI/Swagger documentation.

4.5.1 User Types

The backend has 3 different user types for authentication and authorization.

- Users (standard users) have a username and password. These are accounts are explicitly created by an actual human user. They have an email and a password.
- Anonymous users are created generally without the human user realizing it. It is designed for mobile app users who don't want to explicitly create an account.
- Device users are actual pill organizer devices. They use a shared secret, generated during provisioning, to log in. Their access is restricted to only API resources a device should use.

4.5.2 Authentication

Most API endpoints are authenticated.

- As a standard user, login at `/api/v1/auth/login` with your email and password (JSON).
- As an anonymous user, login at `/api/v1/auth/login_anonymous` with your ID and secret (JSON).
- As a device, login at `/api/v1_2/device/auth` with a protobuf `AuthorizeRequest`.

All of these endpoints return a JWT token. Present a JWT token in the "Authorization" header when making requests. It is your responsibility as an API consumer to store your JWT token securely - the backend doesn't use cookies.

5 App

The app is written in Dart using the Flutter Framework.

5.1 General design

The app generally follows a typical Flutter app directory structure. Here's a brief overview:

- **`/api`** models, repositories, providers, and the API client live in here.
  - `api.dart` is where the API client and DTO classes live.
  - `[...].dart` contain models, repositories, and providers for a particular subset of the API (think devices, medications, etc).
  - `provision.dart` contains all the business logic for provisioning a new device
- **`/l10n`** localization files (for translations) - only English is supported right now
- **`/platform`** utility widgets to implement a behavior that has the native look and feel for either Android or iOS
- **`/proto`** contains the generated protocol buffer files, generated from `pill.proto` (used for Bluetooth sync)
- **`/provider`** providers that aren't related to accessing the API
- **`/screens`** pages in the app
  - `auth/` login/create account/invite code pages
  - `device_settings/medication/medication_entry_wizard.dart` contains medication entry/edit wizard (in this weird folder because this used to be a part of a now-removed device settings page)
  - `my_account/my_account.dart` contains the "my account" page (sign out/sign up)
  - `first_launch.dart` This is the first screen that loads every time a user opens the app. It checks to see if the user is signed in already, immediately switching to the `index` page. If not, it prompts for options (setup new device, sign in, etc).
  - `index.dart` the home page for the app, when a user is signed in. They see their device here.
  - `post_setup_wizard.dart` screens that run after a device has just been provisioned (asking them to create an account, add medications, set times, etc).
  - `provision.dart` screen to guide the user through provisioning (setting up a new device)
- **`/service`** general utility/service classes
- **`/widgets`** general reusable widgets

5.1.1 API client

The app uses retrofit to automatically generate an API client that makes HTTP requests. Client functions are defined declarative using annotations (see retrofit docs).

The client has two filters that intercept every request:

- The `JwtAuthInterceptor` adds the `Authorization: Bearer` token to all HTTP requests (if a token exists). If the token is expired, the interceptor will attempt to sign in

again and acquire another authentication token.
- The `ProblemJsonInterceptor` automatically decodes Problem JSON bodies from failed HTTP requests. Note that due to some bugs in Dio (the underlying HTTP library), this isn't always reliable.

The client is a global variable, access it with `client` (type `RestClient` in `api.dart`).

5.1.2 Repositories & DTOs

Data from the API is received in JSON form as produced by the backend. However, this may not be the most efficient format for us to use the data within the app. JSON data is read in from the API into `[...]DTO` objects, which are then mapped into our actual model object. For example, a schedule is deserialized into a `SimpleScheduleDTO`, which is then mapped into a `SimpleSchedule` object using the `SimpleSchedule.fromDTO` factory (manual process, use something like auto_mappr in the future?). If the structure can be serialized to be sent back to the API, a static method `toDTO` is provided.

Repositories encapsulate the logic for fetching data and converting to and from DTOs. For example, `ScheduleRepository` provides functions to get a dispense time by ID, which fetches the simple schedule from the backend and automatically converts it into a `SimpleSchedule`.

Current best practices for data flow generally have only providers calling repository methods. This best practice was not established until late into the development cycle, so this may not be followed everywhere. Effort should be made to ensure the data flows properly, since direct access to repositories undermines dependency notification.

5.1.3 State management

The project went through a couple of different state management patterns, starting with simple stateful widgets, Bloc, to finally providers using only stateless widgets. There is a mix state management patterns in the codebase, but all modern code should use providers with stateless widgets. There is also a mix of how data flows through providers, this is also an area that is being worked on.

`freezed` is used to enforce immutability.

5.1.4 flutter_esp_ble_prov

The code to provision the WiFi credentials on the firmware is provided by the extension `flutter_esp_ble_prov`. The stock extension does not provide calling a custom endpoint, so we maintain a fork of the extension in the source tree. We call custom endpoints with `customEndpoint`.

Maintaining a separate fork is a maintenance burden, and we should probably just contribute our changes upstream.

5.2 Building, Deploying, and CI/CD

Make sure generated sources (.freezed.dart, .g.dart) are up-to-date with `dart pub run build_runner build`. The app is then built using the standard Flutter toolchain, i.e., `flutter build`.

5.2.1 CI/CD

We use Github Actions to automatically build the app for Android. The iOS app is built in Xcode Cloud. The Android actions runner runs on a Mac Mini located at Brendan's house that has a tendency to go to sleep and stop responding - it is not set up for other developers to use - sorry!

5.2.2 Deploying

5.2.2.1 Apple TestFlight We currently use TestFlight to distribute the app on iOS. There are two testing tracks, internal and external testing. The internal testing list is for development versions and has a small distribution list. The external testing track can be accessed by anyone with a link to join.

Xcode Cloud is configured to automatically distribute builds to internal testing.

5.2.2.2 Google Play Store We currently deploy our app to the Play Store using the Internal and Closed testing tracks. The Internal testing track is for development versions (small distribution list - Design Catapult, Pier-Luc, etc). The Closed testing track has a wider group of testers set up for focus group testing. So the "closed" track is equivalent to our production environment.

The signing key is located at `app/android/app/brendan.jks`, **this should be rekeyed before production** and done properly.

Anyone can join the closed testers list by joining the cabinet-testers Google Group.

6 Firmware

The firmware is built with PlatformIO and esp-idf.

6.1 Project Status

Generally, things work but code quality is poor. The firmware was originally written very fast for rapid development and iteration. There has been a push to refactor out low-quality code but significant time constraints have made this challenging. New features (WiFi, Bluetooth) are well-written, but some other components have hacks upon hacks and inefficient design. The system works "well enough" at present.

6.2 Patches

To work around limitations in the esp-idf, we maintain a series of patches in the firmware source that are applied to PlatformIO's esp-idf. The patches are located in `/firmware/patches/`. We use the python script method based on this PlatformIO document.

Current patches deal with Protocomm NimBLE, adding a few points for our custom Bluetooth code to hook into.

6.2.1 Patch workflow

Patches are automatically applied to the esp-idf when building the project (see above). Note that the patches are (unfortunately) applied to the global esp-idf installation (no known workaround at this time).

Patches are a maintenance burden and should be avoided whenever possible. To create a new patch, Git is the recommended workflow. Checkout the target project in git, make your changes directly to that source tree, and then run `git diff > output.patch` to generate a patchfile. You'll need to modify the python script `apply_patches.py` to include your new patchfile.

6.3 Building

Use `pio run`. Output is in `.pio/build/esp32dev`. Files included in a release are `partitions.bin`, `ota_data_initial.bin`, `firmware.bin` and `bootloader.bin`.

6.3.1 CI/CD

The firmware is automatically built and bundled into a ZIP by Github Actions every time there is a successful push to master. The workflow file is `firmware.yml`.

Note that the firmware is **not** automatically deployed to end-users (this is a manual process, see below).

The latest firmware bundle can be accessed on Github under the "Actions" tab, selecting the run, and downloading the file under "Artifacts".

6.4 Versioning

There are effectively three different version numbers.

1. Build number - the Github Action automatically adds an incrementing number into the `FIRMWARE_BUILD` macro in `config.h`.
2. Revision number - this is the primary major "version" of the firmware, used in OTA firmware updates. Major changes should bump this number in the `FIRMWARE_REVISION` macro in `config.h`. The OTA update system won't accept an update if this version number stays the same.

3. Espressif-defined version, which is a Git hash of the repository when the firmware was built. We don't use this.

The build number is appended to the firmware bundle ZIP file name.

6.5 Manual Flashing

Manual flashing inside PlatformIO is easy, just use the "Upload" task in the PlatformIO pane.

Installing a firmware bundle without PlatformIO requires the ESP flash download tool and a serial-to-USB dongle that supports DTR and RTS (Brendan uses an FTDI-based one). Wire up your dongle to RTS, RXD, TXD, GND, and DTR.

Run the flash download tool in ESP32/develop mode. Configure the partitions as follows:

- `bootloader.bin` - 0x1000
- `firmware.bin` - 0x10000
- `ota_data_initial.bin` - 0xd000
- `partitions.bin` - 0x8000

6.6 Over-the-air updates

We have three mechanisms to deliver OTA firmware updates. All methods are designed to be pushed out/initiated

The first one registers two custom endpoints with protocomm (`fw-version` and `update-fw`). This is the only provisioning method available before provisioning is complete. The `fw-version` endpoint accepts a firmware version and replies with the currently installed firmware version. The `update-fw` endpoint accepts the firmware and begins writing it to the partition This method is unfortunately very slow, but is available before provisioning.

The second one is equivalent to the first, except using our own BLE characteristic (not tied to protcomm). Thus, this method is only available after provisioning is complete.

The last one accepts the firmware via the HTTP POST to `/update`. It is the fastest method, but is only available after provisioning and requires the phone to be on the same WiFi network as the device.

We use the two-slot OTA partitioning method. Note that due to firmware size constraints, **the factory partition is not used and is reclaimed space**. The firmware runs off of one slot and writes incoming OTA updates to the alternate partition. On the next boot, the bootloader attempts to load off the alternate partition and switches back to the original if it can't boot.

6.7 Deploying an update

Deploying a firmware update pushes it out to end-users, making it available for over-the-air firmware updates.

1. Increment the revision number in `config.h` (see above).
2. Build the firmware. You only need `firmware.bin`.
3. Move the `firmware.bin` to `backend/src/main/resources/firmware_latest.net` (overwriting the existing file).
4. Copy the `firmware.bin` again to `backend/src/main/resources/firmware_{REVISION NUMBER}.bin`, making sure the proper revision number is appended to the end of the file name.
5. Edit `backend/src/main/java/jct/pillorganizer/service/FirmwareService.java` and replace the `getLatestVersion` return value to the new revision number.
6. Build and deploy the backend (push to Github and let the actions take care of deployment).

The update will be immediately available to all users, but may take time for all devices to process the update.

7 Servers & Hosting Infrastructure

The backend (and its Postgres database) needs to be hosted somewhere in the cloud. It's cloud-agnostic, we aren't tied to any particular provider and can switch fairly easily.

7.1 Heroku

The backend is currently hosted on Heroku, using their container hosting platform. It is accessible on the web from jctbackend.herokuapp.com. A Docker image is created using Micronaut (see Deploying) and uploaded to Heroku via the Heroku CLI.

This is a managed container hosting service. There shouldn't be any need to do any sort of maintenance, etc. There is an automatic health check that restarts the container if it goes down.

The backend is automatically built and deployed to Heroku on every push to master using Github Actions. See `.github/workflows/backend.yml` for details.

7.1.1 Logs

It may be helpful for debugging to view live logs as it is running in production. In the top left corner of the dashboard, click "More" and select "View Logs".

7.1.2 Database

We use Heroku's managed PostgreSQL addon. The database credentials are automatically injected into our container.

7.2 fly.io

The current tentative plan is to use fly.io for production. Current phase is experimentation, we have no assets currently deployed to fly.io.